William Portfolio — Maintenance & Update Guide

Document version: 1.0 — March 2026
Project: `william-portfolio`
Technology stack: React 19 · TypeScript · Tailwind CSS 4 · Vite · Framer Motion

---

Table of Contents

1. Project Overview
2. Project Structure
3. External Service Dependencies
4. GitHub Pages Deployment
5. Running the Site Locally
6. Updating Content — Section by Section
   - 6.1 Hero Section
   - 6.2 Work Section
   - 6.3 About Section
   - 6.4 Skills Section
   - 6.5 Interests Section
   - 6.6 Contact Section
7. Theming and Visual Design
8. Adding Media to Work Cards
9. Updating Navigation and Social Links
10. Maintenance Checklist

---

1. Project Overview

This is a single-page portfolio site built as a static React application. It is fully client-side — there is no database, no server-side rendering, and no backend to maintain. The only external runtime dependency is the **Web3Forms** API, which handles contact form submissions.

The site uses a custom design system called “Liquid Precision”: Space Grotesk for display headings, Space Mono for labels and code-style elements, and Inter for body text. A WebGL shader in the hero section renders the animated neural web effect. Both light and dark themes are supported and the user’s preference is persisted in `localStorage`.

---

2. Project Structure

All editable source files live under `client/src/`. The table below maps each area of the site to its source file.

Area of the site	Source file
Page layout and section order	`client/src/pages/Home.tsx`
Hero section	`client/src/components/sections/HeroSection.tsx`
Work / projects section	`client/src/components/sections/WorkSection.tsx`
About section	`client/src/components/sections/AboutSection.tsx`
Skills section	`client/src/components/sections/SkillsSection.tsx`
Interests section	`client/src/components/sections/InterestsSection.tsx`
Contact section	`client/src/components/sections/ContactSection.tsx`
Top navigation bar	`client/src/components/TopNav.tsx`
Side nav dots	`client/src/components/NavDots.tsx`
Hero WebGL shader	`client/src/components/ShaderCanvas.tsx`
Global colour tokens	`client/src/lib/theme-tokens.ts`
Theme switching logic	`client/src/contexts/ThemeContext.tsx`
Global CSS and fonts	`client/src/index.css`
HTML head (title, meta, fonts)	`client/index.html`
Build configuration	`vite.config.ts`
Package dependencies	`package.json`

Key design principle

All editable content (project entries, skills, interests, bio text, stats) is stored as plain JavaScript/TypeScript arrays or string constants at the **top of each section file**, clearly separated from the rendering code below. You do not need to understand React to update content — locate the array, edit the values, save the file, and the site rebuilds automatically.

---

3. External Service Dependencies

The site has exactly one external runtime dependency.

Service	Purpose	Where configured	Free tier
Web3Forms	Contact form email delivery	`ContactSection.tsx` line 16	250 submissions/month

The `WEB3FORMS_KEY` constant on line 16 of `ContactSection.tsx` holds your access key (`914ccd03-0492-4897-9fe4-c14bd0178874`). This key is safe to be public in source code — it acts as an alias to your email address, not a secret credential. Submissions are forwarded to `william.tian@gmail.com`.

If you ever need to change the destination email address, create a new key at app.web3forms.com with the new address and replace the constant value.

No other external APIs, CDNs, or paid services are required at runtime. Google Fonts (Space Grotesk, Space Mono, Inter) are loaded from Google's CDN via a `<link>` tag in `client/index.html` — these are free and require no account.

---

4. GitHub Pages Deployment

GitHub Pages can serve a static Vite build directly. The steps below assume you have a GitHub account and `git` installed locally.

Step 1 — Create a repository

Create a new repository at github.com/new. Name it `<your-username>.github.io` for a root domain site, or any name for a project site (e.g., `portfolio`). Set it to **Public**.

Step 2 — Configure Vite for GitHub Pages

If you are deploying to a **project site** (i.e., the URL will be `https://username.github.io/portfolio/`), open `vite.config.ts` and add a `base` property:

```
export default defineConfig({
  base: "/portfolio/",   // replace with your repository name
  // ... rest of config
});
```

If you are deploying to a **root domain site** (`https://username.github.io/`), no change is needed.

Step 3 — Build the site

In your terminal, from the project root:

```
pnpm install        # install dependencies (first time only)
pnpm build          # outputs static files to dist/
```

The `dist/` folder contains the complete static site.

Step 4 — Push to GitHub

```
git init
git remote add origin https://github.com/<your-username>/<repo-name>.git
git add .
git commit -m "Initial portfolio build"
git push -u origin main
```

Step 5 — Enable GitHub Pages

1. Go to your repository on GitHub.
2. Click **Settings → Pages**.
3. Under **Source**, select **GitHub Actions**.
4. GitHub will suggest a workflow — choose **Static HTML** or create the file below.

Step 6 — Add a deployment workflow

Create the file `.github/workflows/deploy.yml` in your project:

```yaml
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pages: write
      id-token: write
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with:
          version: 10
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install
      - run: pnpm build
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist/
      - uses: actions/deploy-pages@v4
```

Push this file to `main`. GitHub Actions will build and deploy automatically on every push. The live URL appears in **Settings → Pages** once the first deployment completes.

Custom domain (optional)

To use a custom domain (e.g., `williamtian.com`):

1. Add a `CNAME` file to `client/public/` containing just your domain name: `williamtian.com`

2. In your DNS provider, add a `CNAME` record pointing your domain to `<your-username>.github.io`.
3. In **Settings → Pages → Custom domain**, enter your domain and enable **Enforce HTTPS**.

---

5. Running the Site Locally

```
# Install dependencies (once)
pnpm install

# Start the development server
pnpm dev
# → Opens at http://localhost:3000 with hot-reload

# Build for production
pnpm build

# Preview the production build locally
pnpm preview
```

Requirements: Node.js 22+ and pnpm 10+. Install pnpm with `npm install -g pnpm` if not already present.

---

6. Updating Content — Section by Section

6.1 Hero Section

File: `client/src/components/sections/HeroSection.tsx`

The hero contains your name, tagline, sub-copy, and the two CTA buttons. All of these are hardcoded strings within the JSX. Search for the text you want to change and edit it in place. The key strings to look for are:

- `"William"` — your display name (large heading)

- `"Bridging design & engineering through rapid prototyping & creative technology."` — the main tagline
- `"I turn ideas into tangible experiences…"` — the sub-paragraph
- `"VIEW WORK"` and `"GET IN TOUCH"` — the two button labels

The `GET IN TOUCH` button scrolls to `#contact`. The `VIEW WORK` button scrolls to `#experience` (the Work section's anchor ID).

The animated neural web background is rendered by `ShaderCanvas`. To disable it entirely, remove the `<ShaderCanvas>` element from `HeroSection.tsx`. To change its colours, see Section 7.

6.2 Work Section

File: `client/src/components/sections/WorkSection.tsx`

All project data lives in the `projects` array starting at line 27. Each entry is a plain object with the following fields:

Field	Type	Description
`id`	string	Unique identifier — do not duplicate
`title`	string	Project name shown on the card
`outcome`	string	One-line result shown as the card subtitle
`tags`	string[]	Discipline/tool pills shown on the card
`context.company`	string	Company or client name (shown as secondary metadata)
`context.role`	string	Your role on the project
`context.period`	string	Year or date range
`type`	string	Category label (e.g., “Product”, “Installation”)
`color`	string	Accent colour for the card top border (OKLCH format)
`featured`	boolean	`true` = full-width card with media block; `false` = standard half-width card
`mediaUrl`	string | null	URL of an image or video to display in featured cards (see Section 8)
`mediaType`	`"image"` | `"video"`	Tells the component how to render the media
`description`	string	Full project description shown in the slide-in detail panel
`process`	string	How you approached the project (shown in the panel)
`outcomes`	string[]	Bullet-point results shown in the panel
`tools`	string[]	Tools used, shown as pills in the panel

To add a new project: copy an existing object, change the `id` to something unique, and fill in the fields. Featured cards (`featured: true`) span the full width and show

the media block; use them for your most visually compelling work. Standard cards sit in the two-column grid.

To remove a project: delete its object from the array.

To reorder projects: the grid renders in array order. Featured cards always span full width regardless of position, so placing them first or last produces the cleanest layout.

6.3 About Section

File: `client/src/components/sections/AboutSection.tsx`

There are two areas to update:

Stats tiles (lines 11–16): the four metric tiles at the bottom of the section.

```
const stats = [
  { value: "5+",  label: "Years Experience" },
  { value: "30+", label: "Prototypes Shipped" },
  { value: "12+", label: "Tools Mastered" },
  { value: "3",   label: "Industries" },
];
```

Replace the `value` and `label` strings with your real numbers.

Bio paragraphs (lines ~106–128): two `<motion.p>` elements contain the body text. Edit the strings directly. The heading (`"Crafting experiences at the intersection of form and function."`) is also a hardcoded string in the `<motion.h2>` above the paragraphs.

6.4 Skills Section

File: `client/src/components/sections/SkillsSection.tsx`

All skills data is in the `skillCategories` array (lines 11–52). Each category has a `category` name and a `skills` array. Each skill has three fields:

Field	Type	Description
`name`	string	Skill label shown next to the bar
`level`	number	Proficiency as a percentage (0–100)
`label`	string	Text label shown at the right end of the bar (e.g., “Expert”, “Advanced”)

The section renders exactly four categories in a two-column grid. If you add a fifth category, the grid will overflow — either replace an existing category or adjust the grid columns in the JSX.

6.5 Interests Section

File: `client/src/components/sections/InterestsSection.tsx`

All interest data is in the `interests` array (lines 12–75). Each entry drives both the grid card and the slide-in detail panel.

Field	Type	Description
`id`	string	Unique identifier
`icon`	string	Single Unicode character used as the card icon
`title`	string	Interest name on the card
`tagline`	string	One-line subtitle on the card
`description`	string	Full paragraph shown in the detail panel
`details`	string[]	Bullet-point list in the detail panel
`tags`	string[]	Pill tags shown in the detail panel
`color`	string	Accent colour for the card border and panel header (OKLCH)

The grid is `3 columns on desktop / 2 on tablet / 1 on mobile`. Six entries fill it evenly; adding a seventh will create an orphaned card on the last row.

6.6 Contact Section

File: `client/src/components/sections/ContactSection.tsx`

Web3Forms key (line 16): your access key is already set. If you ever need to change the destination email, replace this value with a new key from app.web3forms.com.

```
const WEB3FORMS_KEY = "914ccd03-0492-4897-9fe4-c14bd0178874";
```

Social links (lines 18–23): update the `href` values with your real profile URLs.

```
const socialLinks = [
  { label: "LinkedIn", href: "https://linkedin.com/in/YOUR-HANDLE", icon:
"↗" },
  { label: "GitHub",   href: "https://github.com/YOUR-HANDLE",   icon: "↗"
},
  { label: "Dribbble", href: "https://dribbble.com/YOUR-HANDLE", icon: "↗"
},
  { label: "Twitter / X", href: "https://x.com/YOUR-HANDLE",    icon: "↗"
},
];
```

Location text (search for `"Available Remotely"`): update this string if your availability changes.

CTA heading (search for `"Let's build"`): the three-line heading and the sub-paragraph below it are plain strings — edit them to match your current pitch.

Form subject options: the `<select>` dropdown has five options. To add, remove, or rename them, find the `<option>` elements in the JSX and edit their `value` and display text.

---

7. Theming and Visual Design

Light and dark mode

The site defaults to dark mode. The user's choice is saved to `localStorage` under the key `"theme"` and restored on every visit. To change the default theme, open `client/src/App.tsx` and change `defaultTheme="dark"` to `defaultTheme="light"`.

Colour tokens

All colours used across every section are defined in a single file: `client/src/lib/theme-tokens.ts`. The file exports two objects — `dark` and `light` — each containing named colour values. Editing a value here changes it everywhere it is used. The tokens use OKLCH colour notation, which is the standard for Tailwind CSS 4.

Hero shader colours

The WebGL shader in `client/src/components/ShaderCanvas.tsx` has two colour palettes hardcoded in GLSL. The dark-mode palette (mint, violet, teal, ice-blue) is in the `if (u_dark_mode > 0.5)` branch of the `nodeColor()` function. The light-mode palette (bright blue, indigo, sky blue, deep blue) is in the `else` branch. To change the glow colours, edit the `vec3(r, g, b)` values in those branches — values are in the 0.0–1.0 range.

Fonts

The three fonts are loaded in `client/index.html` via a Google Fonts `<link>` tag. To change a font, update both the Google Fonts URL in the `<link>` tag and the `fontFamily` references in the section components and `client/src/index.css`.

---

8. Adding Media to Work Cards

Featured work cards (those with `featured: true` in the `projects` array) display a media block at the top. To activate it:

1. **Prepare your asset.** Supported formats: `.jpg`, `.png`, `.gif`, `.mp4`, `.webm`. For videos, keep clips under 10 seconds and compress to under 5 MB for fast loading.

2. **Host the asset.** GitHub Pages can serve files from your repository, but large media files will slow down the build. The recommended approach is to upload to a free CDN such as Cloudinary (free tier: 25 GB storage) or to your GitHub repository's `client/public/` folder for small images.

3. **Set `mediaUrl` and `mediaType`** in the project object:

```
{
  id: "proj2",
  title: "Kinetic Brand Installation",
  featured: true,
  mediaUrl: "https://your-cdn.com/kinetic-installation-preview.mp4",
  mediaType: "video",
  // ... rest of fields
}
```

The `MediaBlock` component in `WorkSection.tsx` will render a `<video autoPlay loop muted playsInline>` for videos and an `<img>` for images. While `mediaUrl` is `null`, a dashed placeholder is shown.

---

9. Updating Navigation and Social Links

The top navigation bar and the side nav dots are separate components that both list the section labels and anchor IDs. If you rename or reorder sections, update both files.

Component	File	What to edit
Top nav bar	`client/src/components/TopNav.tsx`	`navLinks` array — `label` and `href`
Side nav dots	`client/src/components/NavDots.tsx`	`sections` array — `label` and `id`

The `href` / `id` values must match the `id` attribute on the corresponding `<section>` element. The current mapping is:

Nav label	Section `id`
WORK	`experience`
ABOUT	`about`
SKILLS	`skills`
INTERESTS	`interests`
CONTACT	`contact`

Note that the Work section uses `id="experience"` for historical reasons — this is intentional and does not need to change unless you also update the nav links.

---

10. Maintenance Checklist

Use this checklist when making updates or preparing a new deployment.

Content updates

- [ ] Project entries in `WorkSection.tsx` reflect current and recent work
- [ ] Stats in `AboutSection.tsx` are accurate (years, prototypes, industries)
- [ ] Bio paragraphs in `AboutSection.tsx` reflect current positioning
- [ ] Skill levels in `SkillsSection.tsx` are up to date
- [ ] Social links in `ContactSection.tsx` point to live profiles
- [ ] Contact form tested end-to-end (submit a test message and confirm delivery to Gmail)

Before each deployment

- [ ] Run `pnpm check` (TypeScript check) — should report no errors
- [ ] Run `pnpm build` — should complete without errors
- [ ] Open `dist/index.html` in a browser (or run `pnpm preview`) and check all sections load

- [ ] Test the contact form in the production build
- [ ] Verify the site renders correctly on a mobile device (or using browser DevTools responsive mode)

Periodic maintenance

- [ ] Run `pnpm update` every few months to pull patch-level dependency updates
- [ ] Check web3forms.com dashboard for submission counts — free tier allows 250/month
- [ ] Review Google Fonts availability if fonts stop loading (rare, but the CDN URL may change)

This document was generated alongside the portfolio build in March 2026. For questions about the codebase, refer to the inline comments at the top of each section file.